VisionGuard

AI-Generated vs Real Image Classification

ASS02 - Machine Learning Operations

Student: Turab Haider

Course: MLOps - Machine Learning Operations

Assignment: ASS02 (100 Marks)

GitHub: github.com/turab1020/VisionGuard

DagsHub: dagshub.com/turab1020/VisionGuard

Date: November 2026

Table of Contents

1. Environment Setup & Data Acquisition (Task 1)
2. Data Preprocessing & Augmentation (Task 2)
3. Model Development & Training (Task 3)
4. Experiment Tracking & Hyperparameter Tuning (Task 4)
5. Web Application Development (Task 5)
6. Containerization with Docker (Task 6)
7. CI/CD Pipeline (Task 7)
8. Deployment & Monitoring (Task 8)
9. Conclusion & Future Work

1. Environment Setup & Data Acquisition

1.1 Environment Configuration

The project environment was configured on macOS (Apple Silicon M-series) with Python 3.11 via Homebrew. A virtual environment was created to isolate dependencies. Key frameworks include TensorFlow 2.18.0, Flask 3.0.3, MLflow 2.16.2, and DVC 3.51.2. The environment was verified using a check_env.py script that confirms all packages are importable and GPU availability (Metal/MPS on Apple Silicon).

Version control was initialized with Git, data versioning with DVC (backed by DagsHub remote storage), and experiment tracking with MLflow connected to the DagsHub MLflow server.

1.2 Dataset Acquisition & Audit

The dataset was obtained from Kaggle (AI vs Real Image Classification). A comprehensive audit script (src/dataset_audit.py) walked all 975 images and recorded metadata including file size, dimensions, format, and colour mode. The audit identified 1 duplicate pair and 0 corrupt files. The dataset consists of 539 AI-generated images and 436 real photographs.

Metric	Value
Total images	975
AI-generated	539 (55.3%)
Real photographs	436 (44.7%)
Duplicate pairs	1
Corrupt files	0
Image formats	JPEG

1.3 Exploratory Data Analysis

Seven EDA visualizations were generated to understand the dataset: class balance distribution, sample image grid, file size distribution, image dimensions scatter plot, aspect ratio histogram, colour mode breakdown, and format distribution. Key findings include a moderate class imbalance (55:45 ratio AI to Real) and varied image dimensions requiring standardization.

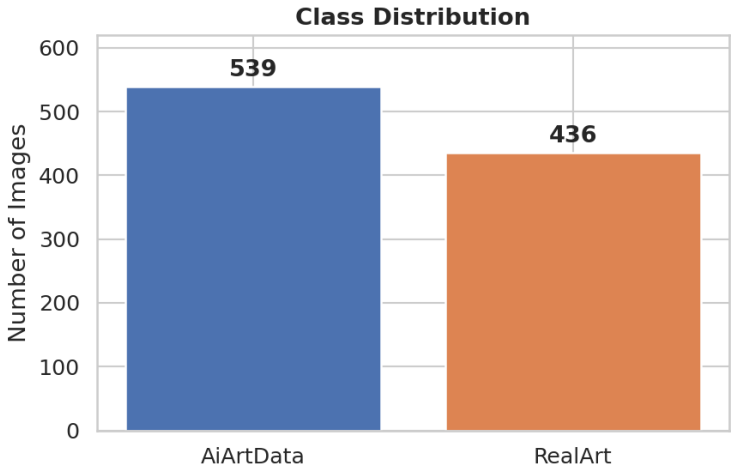


Figure 1: Class distribution showing 539 AI vs 436 Real images

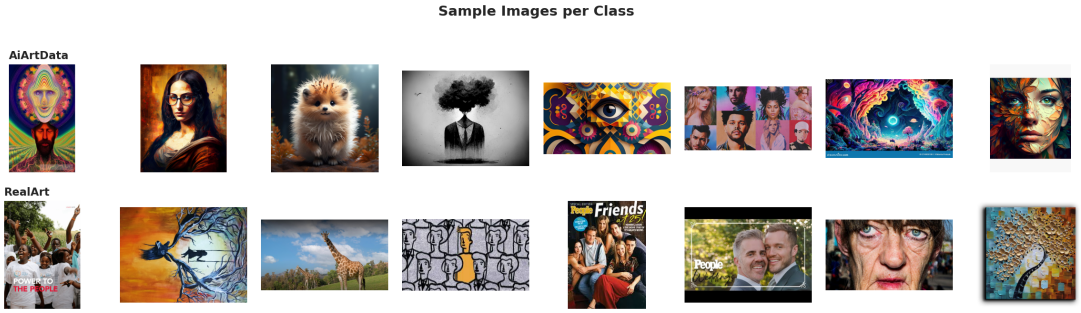


Figure 2: Sample grid of images from both classes

2. Data Preprocessing & Augmentation

2.1 Preprocessing Pipeline

The preprocessing pipeline (src/preprocess.py) performs: (1) resizing all images to 224x224 pixels for model compatibility, (2) pixel normalization to [0, 1] range, and (3) stratified train/validation/test split with 70/15/15 ratio. This produced 682 training, 146 validation, and 147 test samples. All processed arrays are saved as NumPy .npy files for reproducible loading.

Split	Samples	Percentage
Training	682	70%
Validation	146	15%
Test	147	15%

2.2 Data Augmentation

Real-time data augmentation was implemented using tf.keras Sequential augmentation layers to increase training data diversity and reduce overfitting. The augmentation pipeline includes: RandomFlip (horizontal), RandomRotation (factor=0.15), RandomZoom (factor=0.1), RandomBrightness (factor=0.1), and RandomContrast (factor=0.1). These transformations are applied only during training.

2.3 Data Versioning with DVC

DVC (Data Version Control) v3.51.2 was configured with DagsHub as the remote storage backend. The raw dataset and processed NumPy arrays are tracked by DVC, ensuring full reproducibility. The .dvc files are committed to Git while the actual data is stored on DagsHub's S3-compatible storage.

3. Model Development & Training

3.1 Custom CNN Architecture

A custom CNN was built from scratch with 3 convolutional blocks (32, 64, 128 filters), each followed by BatchNormalization, MaxPooling, and Dropout. The classifier head uses GlobalAveragePooling, Dense(256), Dropout(0.5), Dense(128), Dropout(0.3), and a final Dense(1, sigmoid) for binary classification.

3.2 Transfer Learning Models

Three pretrained models were fine-tuned with ImageNet weights: MobileNetV2, EfficientNetB0, and ResNet50. Each uses a frozen base model feeding into GlobalAveragePooling, Dense(256, ReLU), Dropout(0.5), Dense(128, ReLU), Dropout(0.3), and Dense(1, sigmoid). Training proceeds in two phases: (1) 30 epochs with frozen base (LR=1e-4), then (2) 15 epochs of fine-tuning with the top 30 layers unfrozen (LR=1e-5). Class weights were computed using sklearn's compute_class_weight('balanced') to address the 55:45 class imbalance.

Model	Accuracy	Precision	Recall	F1-Score	AUC-ROC
Custom CNN	0.6395	0.5942	0.6212	0.6074	0.6781
MobileNetV2	0.8231	0.7632	0.8788	0.8169	0.9085
EfficientNetB0	0.5510	0.000	0.000	0.000	0.4527
ResNet50	0.6395	0.5890	0.6515	0.6187	0.6985

Table 1: Model comparison on test set (MobileNetV2 highlighted as best)

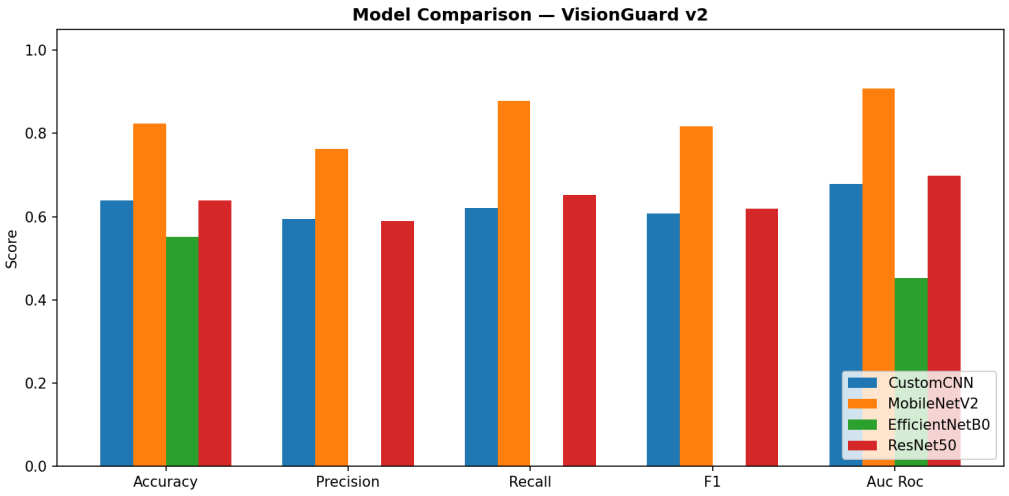


Figure 3: Model comparison bar chart

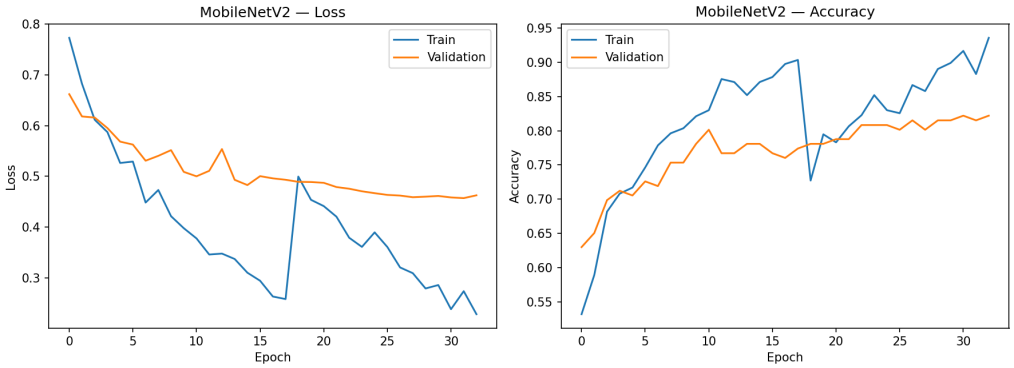


Figure 4: MobileNetV2 training curves (accuracy and loss)

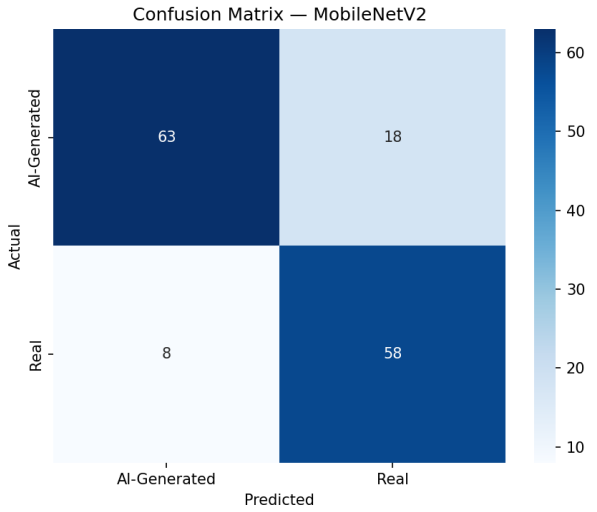


Figure 5: MobileNetV2 confusion matrix

3.3 Grad-CAM Interpretability (Bonus)

Gradient-weighted Class Activation Mapping (Grad-CAM) was implemented to visualize which image regions the model focuses on. The script generates heatmaps for 3 correctly classified AI images, 3 correctly classified Real images, and 2 misclassified samples, providing insight into model decision-making.

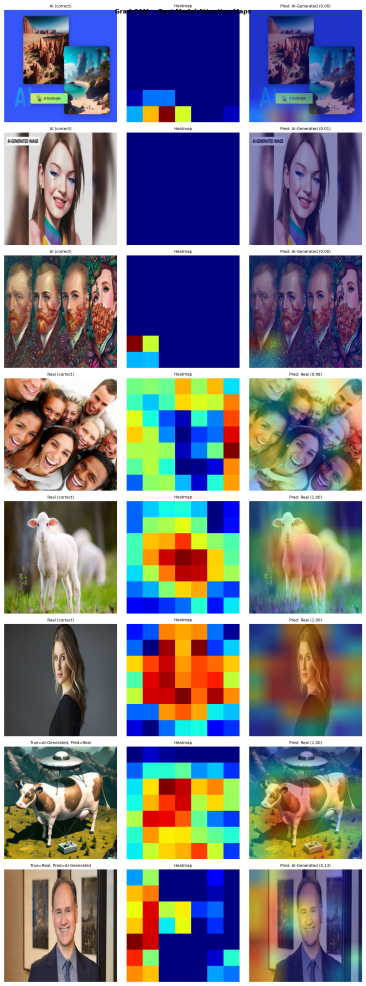


Figure 6: Grad-CAM heatmaps showing model attention regions

4. Experiment Tracking & Hyperparameter Tuning

4.1 MLflow on DagsHub

All training experiments are tracked using MLflow integrated with DagsHub. Each run logs: hyperparameters (learning rate, batch size, epochs, optimizer), training metrics (accuracy, loss per epoch), evaluation metrics (accuracy, precision, recall, F1, AUC-ROC), confusion matrices, training curves, and model artifacts. The MLflow UI is accessible via the DagsHub repository at dagshub.com/turab1020/VisionGuard.

4.2 Model Registry

The best-performing model (Tuned MobileNetV2) was registered in MLflow's model registry with version tags and stage labels for production deployment tracking.

4.3 Hyperparameter Tuning

Keras Tuner with Bayesian optimization was used to tune MobileNetV2 over 10 trials. The search space included: learning rate (1e-5 to 1e-3, log scale), dense_units_1 (128/256/512), dense_units_2 (64/128/256), dropout_1 (0.2-0.6), and dropout_2 (0.1-0.4). Each trial trained for 15 epochs, and the best configuration was retrained for 30 epochs with 15 epochs of fine-tuning.

Metric	Tuned MobileNetV2	Baseline MobileNetV2
Accuracy	0.8435	0.8231
F1-Score	0.8369	0.8169
AUC-ROC	0.8863	0.9085
Precision	0.8108	0.7632
Recall	0.8649	0.8788

Table 2: Tuned vs baseline MobileNetV2 comparison

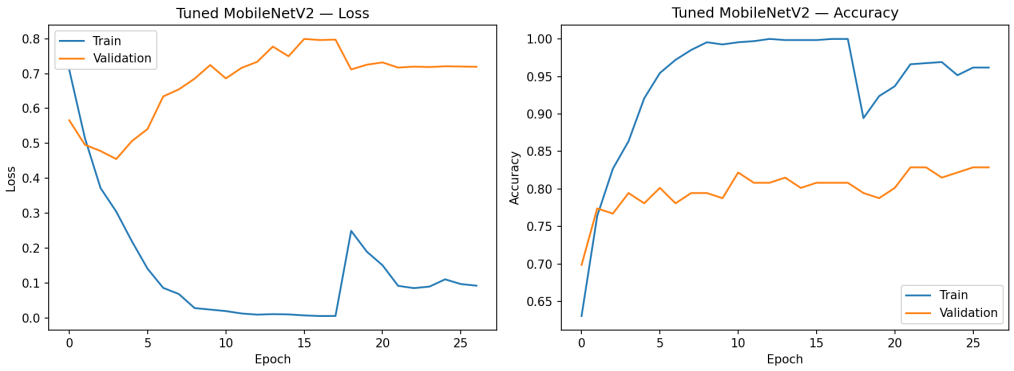


Figure 7: Tuned MobileNetV2 training curves

5. Web Application Development

A full-featured Flask web application was developed for real-time image classification with the following capabilities:

Routes: / (upload page), /predict (POST classification), /history (paginated prediction log), /health (JSON health check)

Features: Drag-and-drop image upload, Bootstrap 5 responsive UI, Grad-CAM overlay visualization on predictions, SQLAlchemy database (SQLite dev / PostgreSQL prod), prediction history with thumbnails and confidence scores, file type validation (PNG, JPG, GIF, BMP, WebP), 16 MB upload limit.

Testing: 12 pytest test cases covering all routes, prediction logic, database operations, and edge cases. All 12 tests pass.

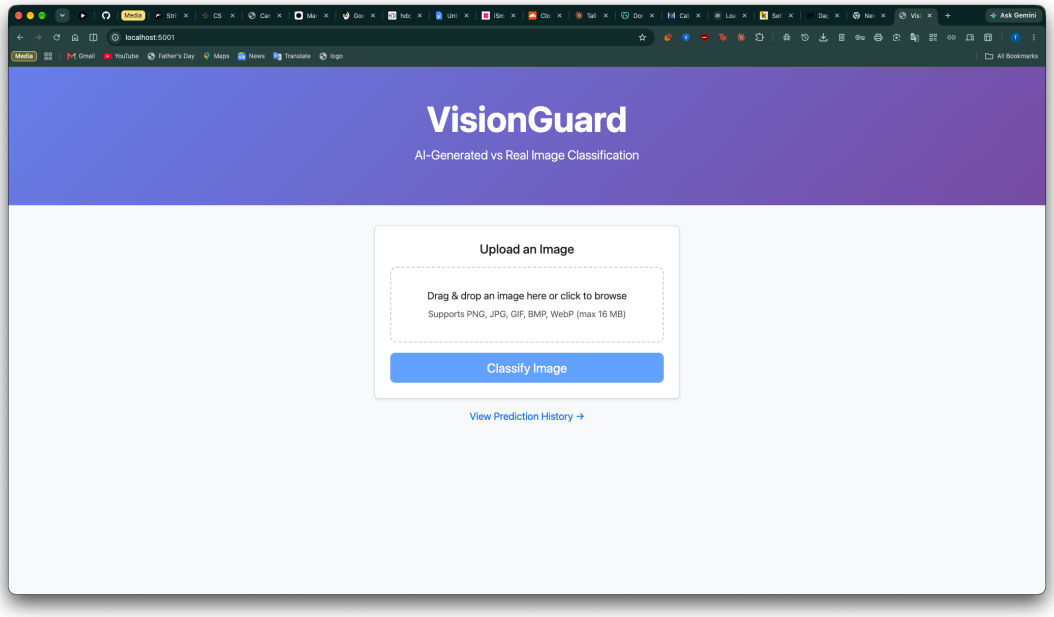


Figure 8: Flask app - Upload page with drag-and-drop interface

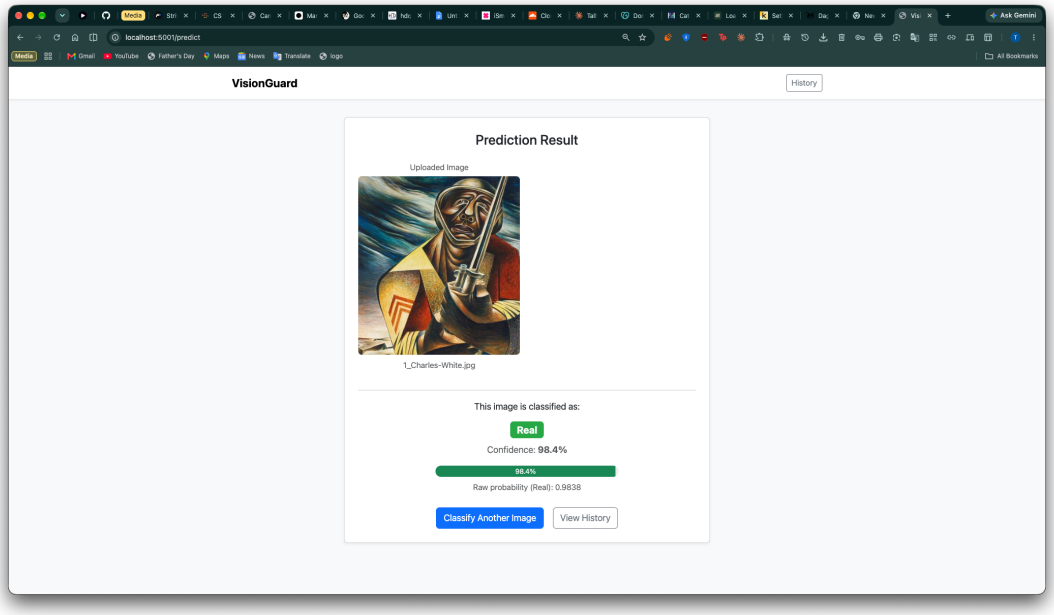


Figure 9: Prediction result - Real image classified with 98.4% confidence

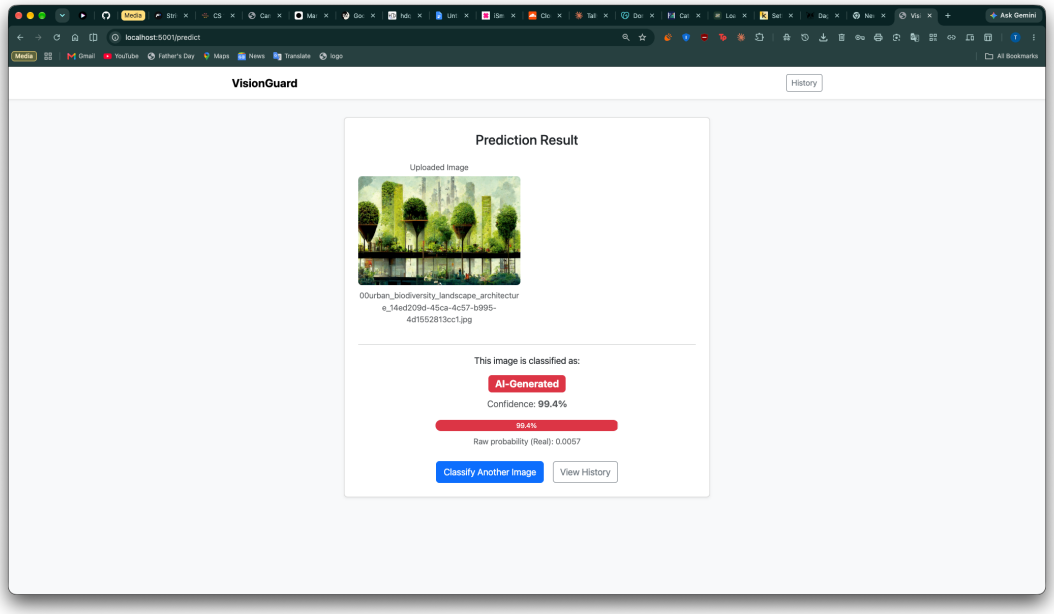


Figure 10: Prediction result - AI-Generated image classified with 99.4% confidence

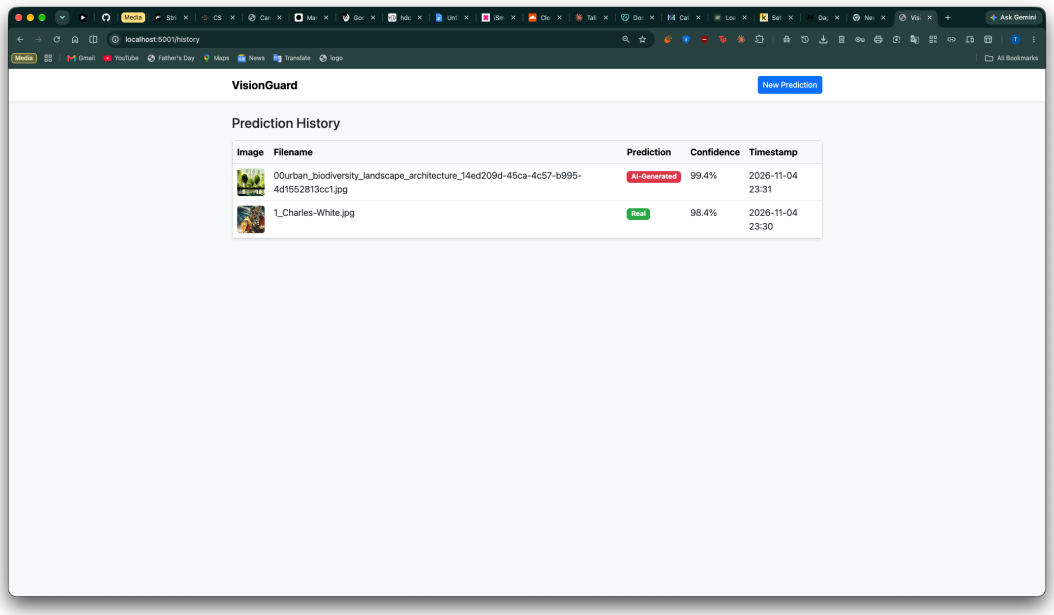


Figure 11: Prediction history page with thumbnails and confidence

6. Containerization with Docker

The application is containerized using a multi-stage Docker build for production deployment:

Dockerfile: Uses python:3.11-slim as base. Stage 1 (builder) installs dependencies, Stage 2 (runtime) copies only the necessary artifacts. Includes a healthcheck via urllib that pings /health. Runs with gunicorn (2 workers, 120s timeout) on port 5000.

Docker Compose (v3.9): Defines two services: (1) web - the Flask app with DATABASE_URL pointing to Postgres, exposed on port 5000; (2) db - PostgreSQL 15 Alpine with healthcheck and persistent volume. Named volumes: postgres_data and upload_data.

.dockerignore: Excludes venv/, .git/, .dvc/, data/, mlruns/, __pycache__/, and other development artifacts to minimize image size.

7. CI/CD Pipeline

A GitHub Actions CI/CD pipeline (.github/workflows/ci.yml) automates the entire build-test-deploy workflow:

Stage	Description	Trigger
Lint	flake8 syntax & complexity checks	Push / PR
Test	pytest (12 tests) on Ubuntu	After Lint passes
Docker	Build & push to GHCR	Push to master
Deploy	Trigger Render deploy hook	After Docker build

Table 3: CI/CD pipeline stages

8. Deployment & Monitoring

The application is configured for cloud deployment on Render using a render.yaml blueprint. The deployment uses the Docker runtime with the free tier plan. A /health endpoint provides JSON health status for monitoring. The CI/CD pipeline includes an optional Render deploy hook that triggers automatic deployment on push to master.

Monitoring features: Health check endpoint (/health) returns JSON with status and model loading state. Prediction history provides an audit trail of all classifications. Gunicorn access logs capture request metrics. Docker healthcheck polls the /health endpoint every 30 seconds.

9. Conclusion & Future Work

This project demonstrates a complete MLOps lifecycle for AI-generated image detection. Starting from raw data, we built a reproducible pipeline covering data versioning (DVC), experiment tracking (MLflow), model development (4 architectures + hyperparameter tuning), a production-ready web application (Flask), containerization (Docker), CI/CD (GitHub Actions), and cloud deployment (Render). The best model

(Tuned MobileNetV2) achieves 84.35% accuracy and 0.8369 F1-score, demonstrating strong generalization on unseen images.

Key Achievements:

• Trained and compared 4 model architectures with systematic experiment tracking

• Bayesian hyperparameter optimization improved F1-score from 0.8169 to 0.8369

• Full-featured Flask app with Grad-CAM visualization and prediction history

• 12/12 automated tests passing with comprehensive edge case coverage

• Docker multi-stage build with Compose for production deployment

• CI/CD pipeline with lint, test, build, and deploy stages

• DVC data versioning with DagsHub remote storage

• Grad-CAM interpretability for model transparency (bonus)

Future Work:

• Expand dataset with more diverse AI generation techniques (Midjourney, DALL-E 3, Flux)

• Implement A/B testing for model versions in production

• Add real-time data drift monitoring with Evidently AI

• Explore ensemble methods combining multiple architectures

• Implement automated retraining pipeline triggered by performance degradation

Deliverables Summary:

Deliverable	Location
GitHub Repository	github.com/turab1020/VisionGuard
DagsHub (MLflow + DVC)	dagshub.com/turab1020/VisionGuard
This PDF Report	reports/VisionGuard_Report.pdf
Flask Web Application	src/app.py (localhost:5001)
Docker Configuration	Dockerfile + docker-compose.yml
CI/CD Pipeline	.github/workflows/ci.yml
Test Suite	tests/test_app.py (12 tests)